Early Warning of Food Price Spikes in Sri Lankan Markets

Hands-on Activity: ML Model Deployment & Comparison (Random Forest, XGBoost, LightGBM, CatBoost, with SHAP analysis)

Name: [YOUR NAME] Index number: [YOUR INDEX NUMBER] Course: [COURSE CODE]
Date: [DATE]

1. Problem definition

Food is a large part of household spending in Sri Lanka, and prices can change sharply from one month to the next. Sudden rises hurt low-income families most, and the 2022 economic crisis showed how quickly food prices can climb. If a rise can be anticipated, households, traders and the authorities have time to respond (for example by releasing stocks, adjusting imports or targeting support).

This activity builds an early-warning model that answers one question: will the retail price of a food item, in a given Sri Lankan market, rise by 10% or more next month? This is a binary classification task (spike = yes or no). Four ensemble models are trained, compared and the best one is selected. SHAP is then used to explain what drives the predictions, and the selected model is deployed behind a FastAPI backend and a Streamlit dashboard.

2. Dataset

Source: retail food prices collected by the UN World Food Programme (WFP) and published on the Humanitarian Data Exchange (https://data.humdata.org/dataset/wfp-food-prices-for-sri-lanka). The file used has 32,496 price records for 44 markets and 42 items, in Sri Lankan rupees (LKR), from January 2004 to September 2025. Items include rice, wheat flour, sugar, vegetables, fruit, fish, eggs, pulses and fuel.

Coverage is thin before mid-2023 (a few hundred records per year in most years) and dense from June 2023 (about 1,000 records every month), as Figure 1 shows.

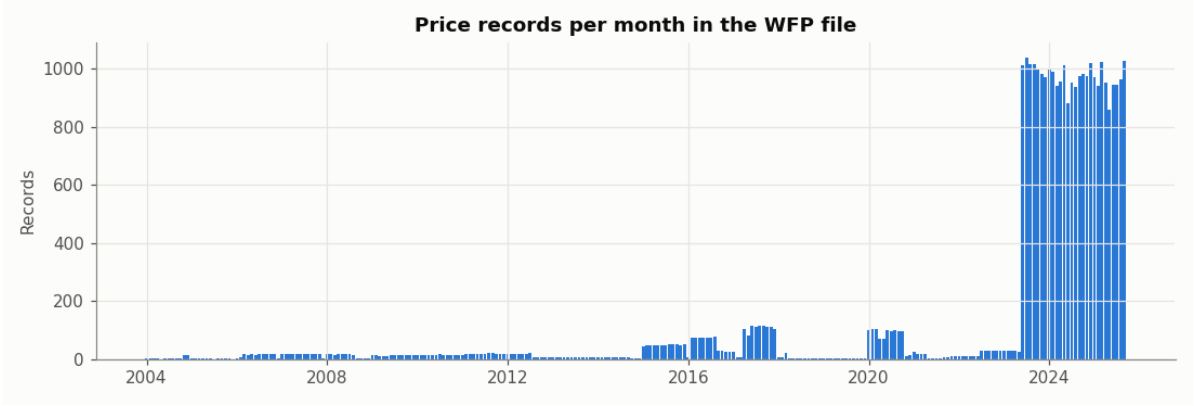


Figure 1: Price records per month in the WFP file

2.1 Cleaning

- Kept retail prices of food items in real markets. Removed 194 "National Average" rows (no location), wholesale prices and fuel (non-food).
- Removed items with fewer than 100 records, and kept one record per market, item and month. This leaves 31,246 records, 37 items and 42 markets.
- Kept only records with a full 3-month price history for the same market and item (needed for the features), giving 22,575 rows. About 19% of them are spikes.

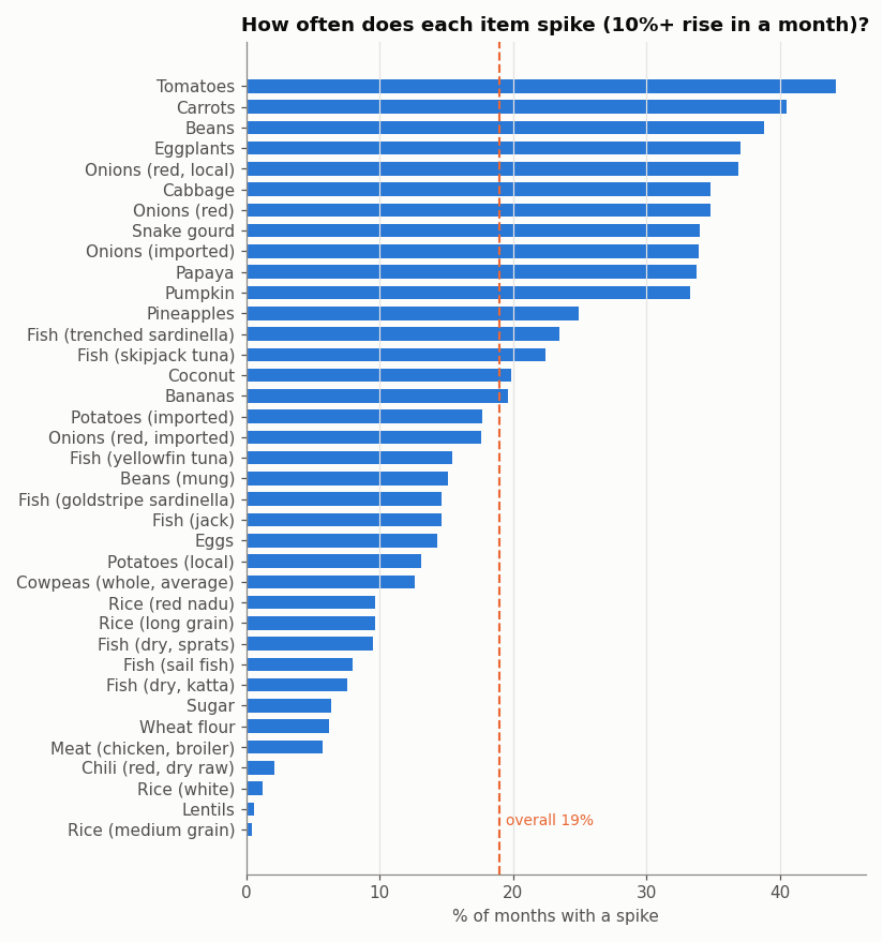


Figure 2: How often each item rises 10% or more in a month

Figure 2 shows that vegetables such as tomatoes, carrots and eggplant spike often, while staples such as rice, sugar and wheat flour rarely do. Because "which item is it" is such a strong clue, a baseline that simply uses each item's usual spike rate is included to check what the models add.

3. Methodology

3.1 Features (past information only)

All features use prices up to the latest known month, never the month being predicted:

- price_now: latest price (LKR); change_1m and change_2m: price change over the last two months; vs_3m_avg: latest price compared with its 3-month average.
- national_avg_now and national_change_1m: the latest national average for the item and how it moved; market_vs_national: whether this market is dearer or cheaper than the national average.
- month (season), latitude, longitude, province (9 columns) and item (37 columns, one per item). In total 56 features.

Target: spike = 1 if next month's price is at least 10% higher than the latest price.

3.2 Splitting by time

- Train: target months up to June 2024 (11,163 rows, 19.4% spikes). Models learn and are tuned here.
- Validation: July to December 2024 (4,527 rows, 20.8% spikes). Used to choose the best model and the alert threshold.
- Test: January to September 2025 (6,884 rows, 17.1% spikes). Used once for the final scores.
- One row was held out for the inference demo (Tomatoes, Colombo City, September 2025) and removed from all three sets.

The split is by time, not random, because a real early-warning system only sees the past. A random split would let the model learn from months after the ones it is tested on.

3.3 Models and tuning

- Random Forest: many trees trained independently, predictions averaged.
- XGBoost: trees built one after another, each correcting the errors of the previous ones.
- LightGBM: a faster boosting library that grows trees leaf by leaf.
- CatBoost: a boosting library with built-in protection against over-fitting.
- Each model was tuned with RandomizedSearchCV and 3-fold TimeSeriesSplit (each fold validates on a period after its training data), scored by ROC-AUC, on the training period only.
- Baseline: each item's spike rate in the training period.

3.4 Evaluation metrics

- ROC-AUC: the chance that a real spike is ranked above a non-spike (0.5 = guessing). Used to choose the best model, because it does not depend on a threshold.
- PR-AUC (average precision): how well spikes are found while keeping false alarms low. Guessing gives about the spike rate (0.17 on the test set).
- Precision, recall and F1 at an alert threshold chosen on the validation set. Plain accuracy is not used: always predicting "no spike" would be about 83% accurate and useless.

4. Results

Model	Val AUC	Test AUC	Test PR-AUC	Precision	Recall	F1	Tuning time (s)
LightGBM	0.800	0.692	0.282	0.274	0.608	0.378	12.5
XGBoost	0.794	0.691	0.287	0.265	0.564	0.360	14.4
Random Forest	0.778	0.672	0.263	0.253	0.622	0.360	27.0
CatBoost	0.776	0.676	0.282	0.254	0.578	0.353	18.2
Baseline: item	0.711	0.664	0.248	0.280	0.518	0.363	-

spike rate							

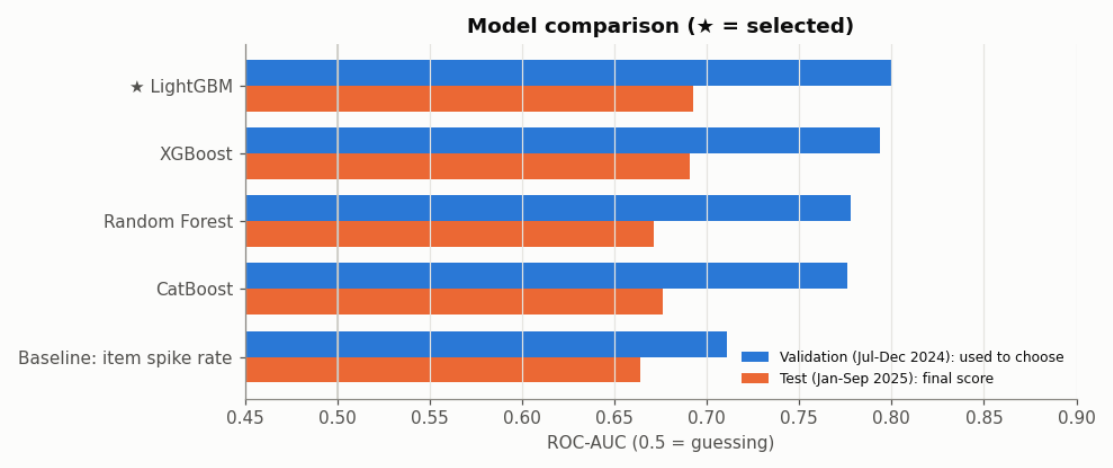


Figure 3: ROC-AUC on validation (used to choose) and test (final score)

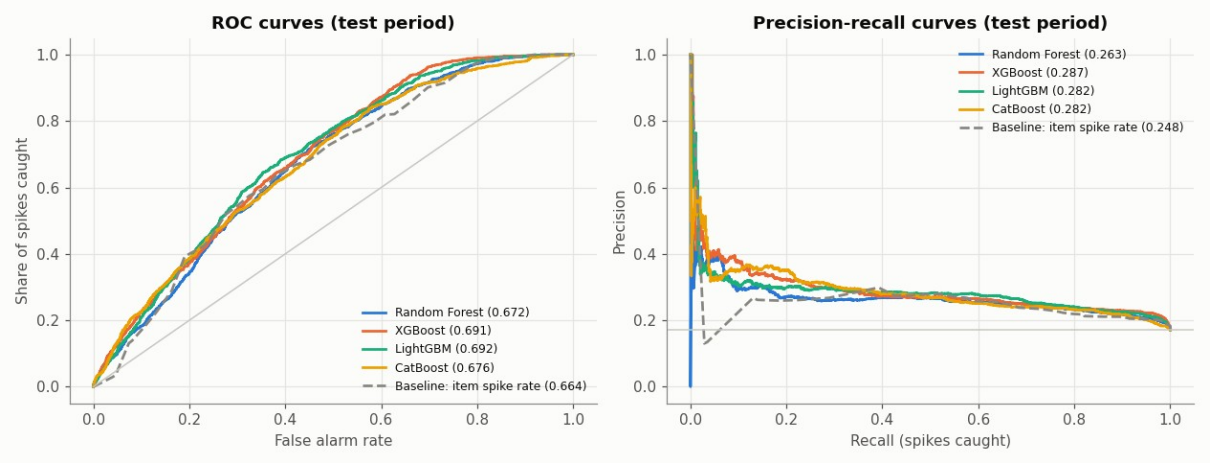


Figure 4: ROC and precision-recall curves on the test period

4.1 Model selection

LightGBM had the highest validation ROC-AUC (0.800), with XGBoost very close (0.794), so LightGBM was selected. On the test period it scored ROC-AUC 0.692, PR-AUC 0.282 (the spike rate is 0.171), precision 0.274 and recall 0.608.

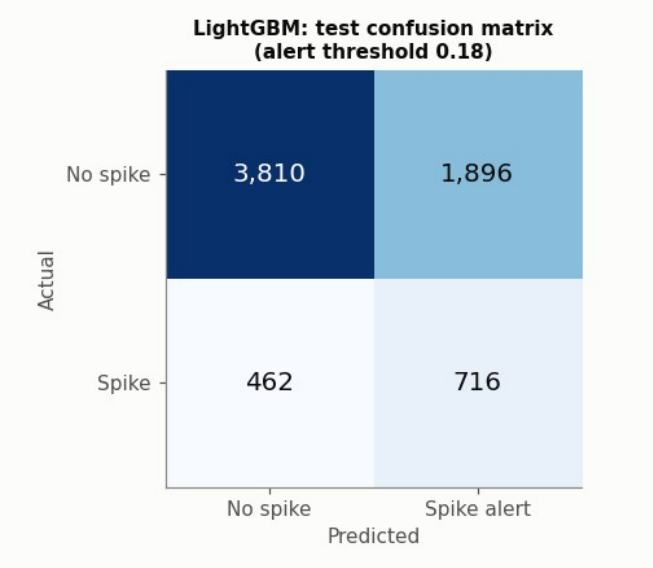


Figure 5: LightGBM confusion matrix on the test period

At the chosen threshold the model raised 2,612 alerts in the test period. 716 of them were real spikes (precision 0.27), and it caught 716 of the 1,178 real spikes (recall 0.61). In other words it catches about 6 in 10 spikes, at the price of roughly 3 false alarms for every real spike found.

4.2 Is the best model really better?

A bootstrap over whole test months (1,000 resamples) was used to see whether the ROC-AUC gaps could be chance:

Comparison (A vs B)	AUC(A) - AUC(B)	95% range	Clear difference?
LightGBM vs Baseline	0.031	-0.038 to 0.109	no
LightGBM vs Random Forest	0.021	-0.008 to 0.045	no
LightGBM vs XGBoost	0.002	-0.012 to 0.016	no
LightGBM vs CatBoost	0.016	-0.008 to 0.043	no

- The four models are very close. LightGBM and XGBoost are practically tied (test AUC 0.692 vs 0.691), and no gap between the models has a 95% range that excludes zero. I therefore do not claim LightGBM is truly better than the others; it simply had the best validation score, which was the rule set in advance.
- All four models beat the baseline on test ROC-AUC (0.664 for the baseline) and PR-AUC, but only by a small margin, and the bootstrap range for LightGBM vs baseline includes zero. Much of the predictable part of a spike is simply "which item is it", which the baseline already captures. The models add some skill beyond that, but modestly.

- Spikes are partly random (weather, supply shocks, reporting noise), so a perfect early-warning model is not realistic. The value of the model lies more in a ranked risk score and in the explanations below than in exact predictions.

5. SHAP analysis

SHAP (SHapley Additive exPlanations) explains a prediction by giving each feature a value: how much it pushed the spike risk up or down compared with the average prediction. It was computed for the selected LightGBM model on 2,500 test rows. For LightGBM the values are in log-odds (positive = higher spike risk). Item and province are stored as many 0/1 columns, so they are also added back together.

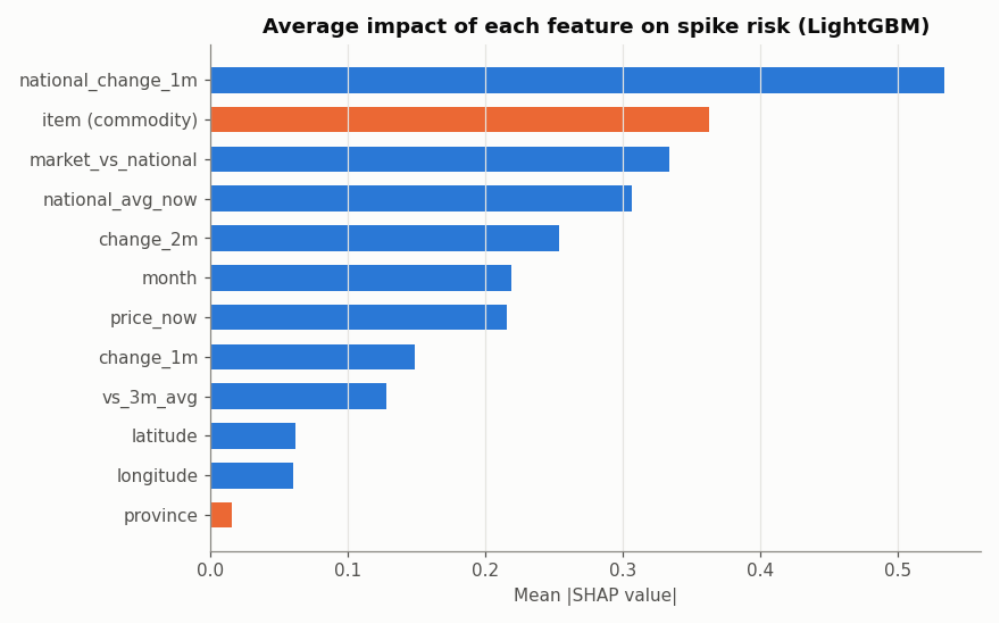


Figure 6: Average impact of each feature on spike risk

The largest overall influence comes from the national price trend (national_change_1m, about 20% of the total impact), then which item it is (about 14%), how the market compares with the national price (about 13%), the national price level (about 12%), the price change in the month before last (about 10%) and the calendar month (about 8%).

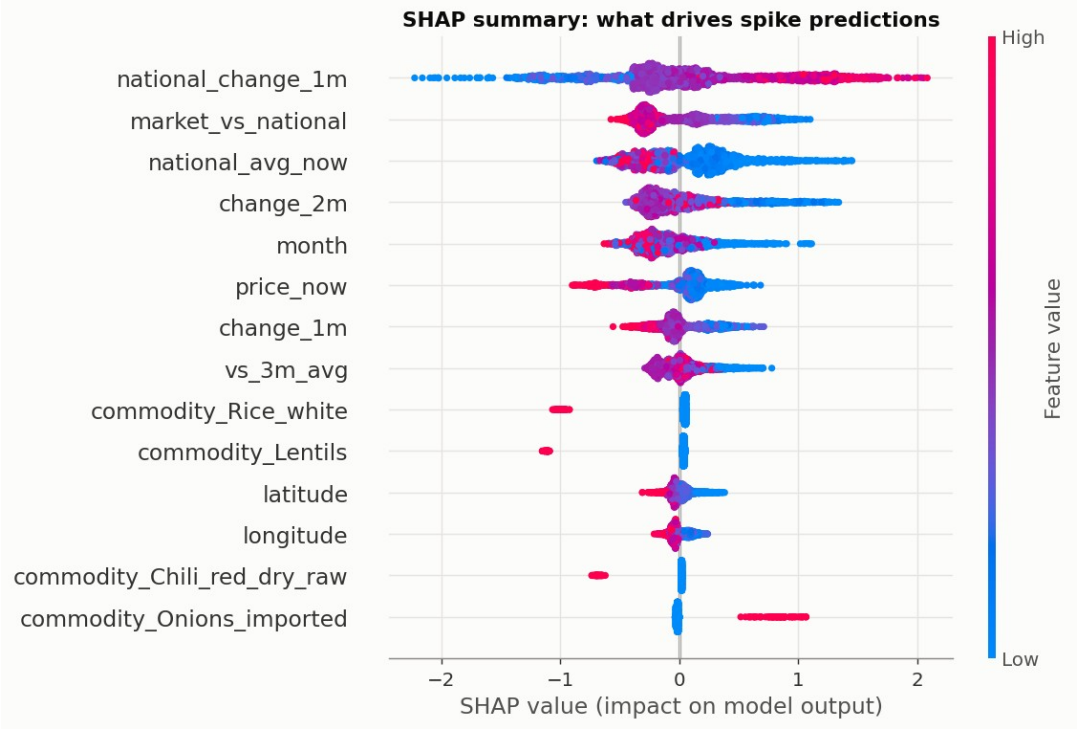


Figure 7: SHAP summary (each dot is one test row; red = high feature value)

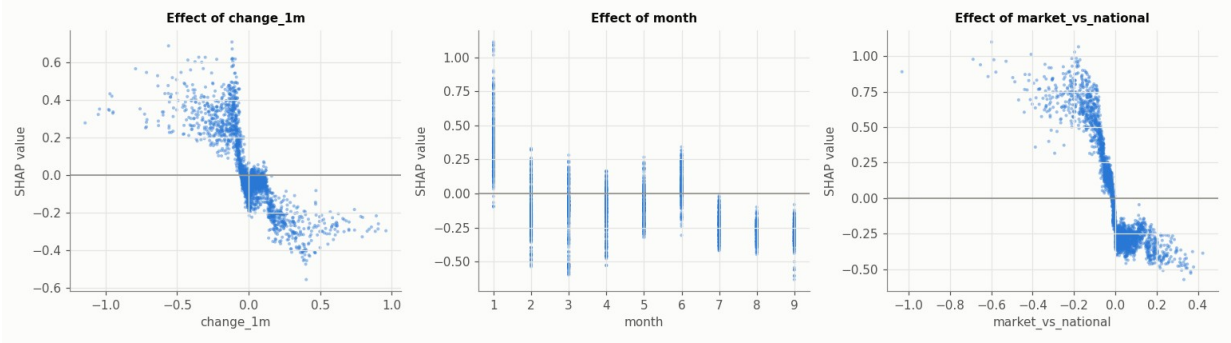


Figure 8: Shape of the effect of three features

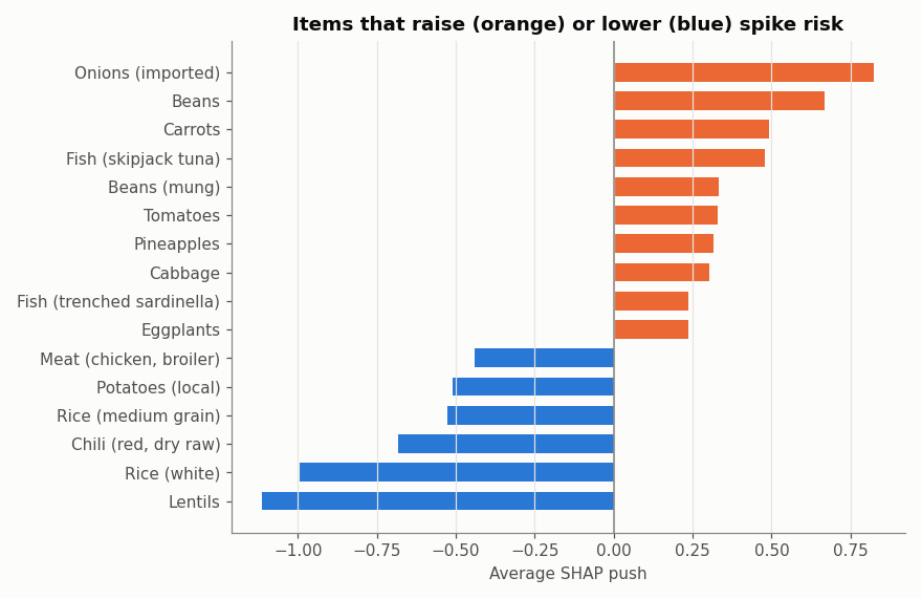


Figure 9: Items that raise or lower spike risk

5.1 What the SHAP results say

- Mean reversion: when an item's price has just fallen (negative change_1m or change_2m), the risk of a spike next month goes up; after a rise, the risk goes down. Prices that dropped sharply tend to bounce back.
- Catch-up effect: a market that is cheaper than the national average (negative market_vs_national) has a higher spike risk; a dearer market has a lower risk.
- National momentum: when the national average price rose last month, spike risk is higher, so price pressure spreads across markets.
- Item type: imported onions, beans, carrots and skipjack tuna carry the highest risk; lentils, white rice, red chili and local potatoes carry the lowest. This matches Figure 2.
- Season: January shows the highest risk. The test period covers only January to September 2025, so effects of October to December cannot be judged from the test set.
- Cheap items (low national price) spike more than expensive ones, which reflects how volatile cheap vegetables are.

5.2 Explaining one prediction

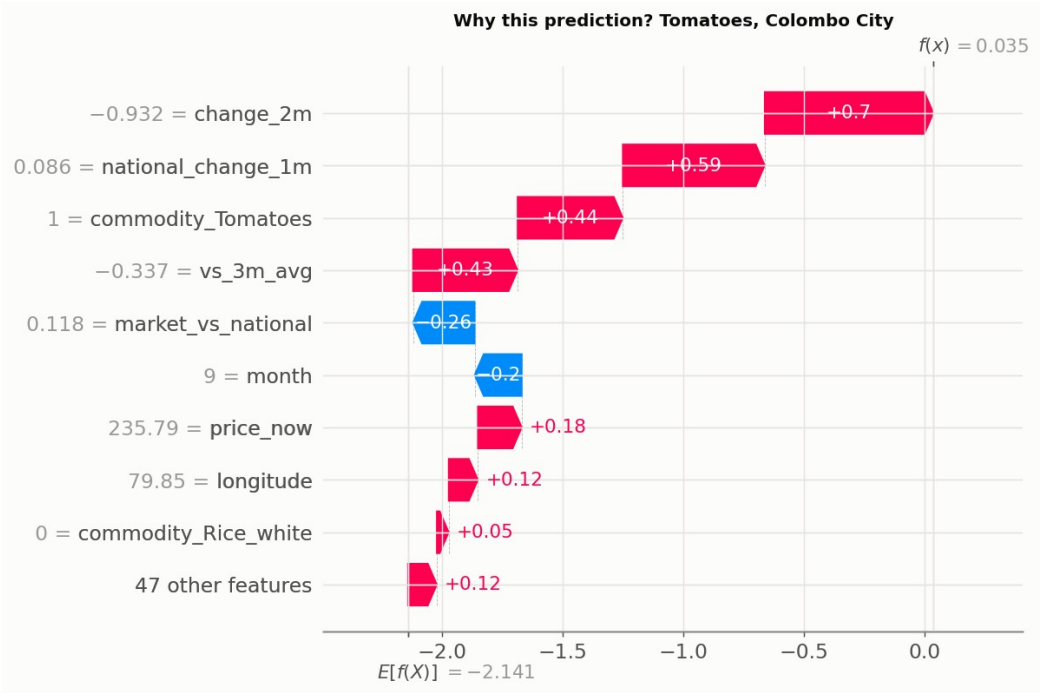


Figure 10: Waterfall for the held-out example (Tomatoes, Colombo City, September 2025)

For the held-out example, the latest price was 235.79 LKR after 213.29 and 541.56 LKR in the two months before. The deployed model gave a spike probability of about 45%, so it raised an alert. The main drivers were the item (tomatoes are volatile), the national price rise, and the large fall in the price two months earlier (mean reversion). The month (September) pushed the risk down. The actual price rose only to 243.66 LKR (+3.3%), so this alert was a false alarm. That is an honest illustration of the precision figure above.

6. System architecture and inference

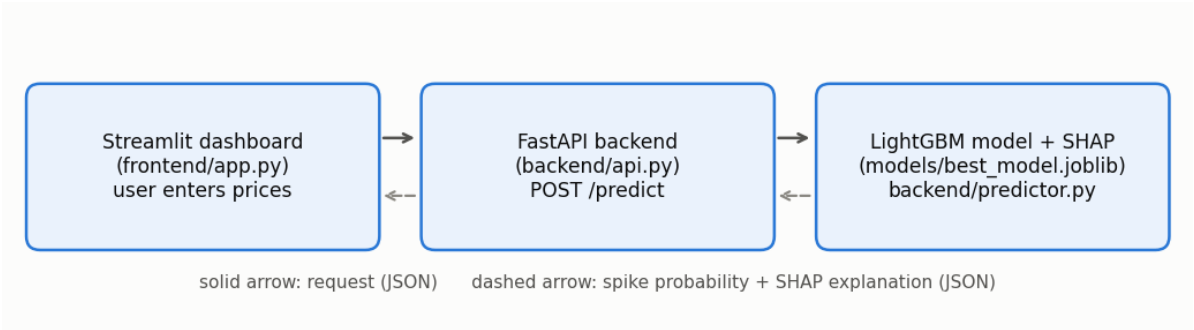


Figure 11: Application architecture

- Model: LightGBM was retrained on all rows (excluding the held-out row) with its tuned settings and saved with joblib.

- Backend (FastAPI): backend/api.py exposes GET /health, GET /model-info, GET /markets, GET /commodities and POST /predict. Input is validated with Pydantic and a SHAP explanation is returned with each prediction.
- Frontend (Streamlit): frontend/app.py lets the user choose an item, a market and a month, enter the last three prices and the national averages, and see the spike probability, the alert, and a SHAP bar chart explaining the result. It also has tabs for the model comparison, the SHAP charts and a price explorer.

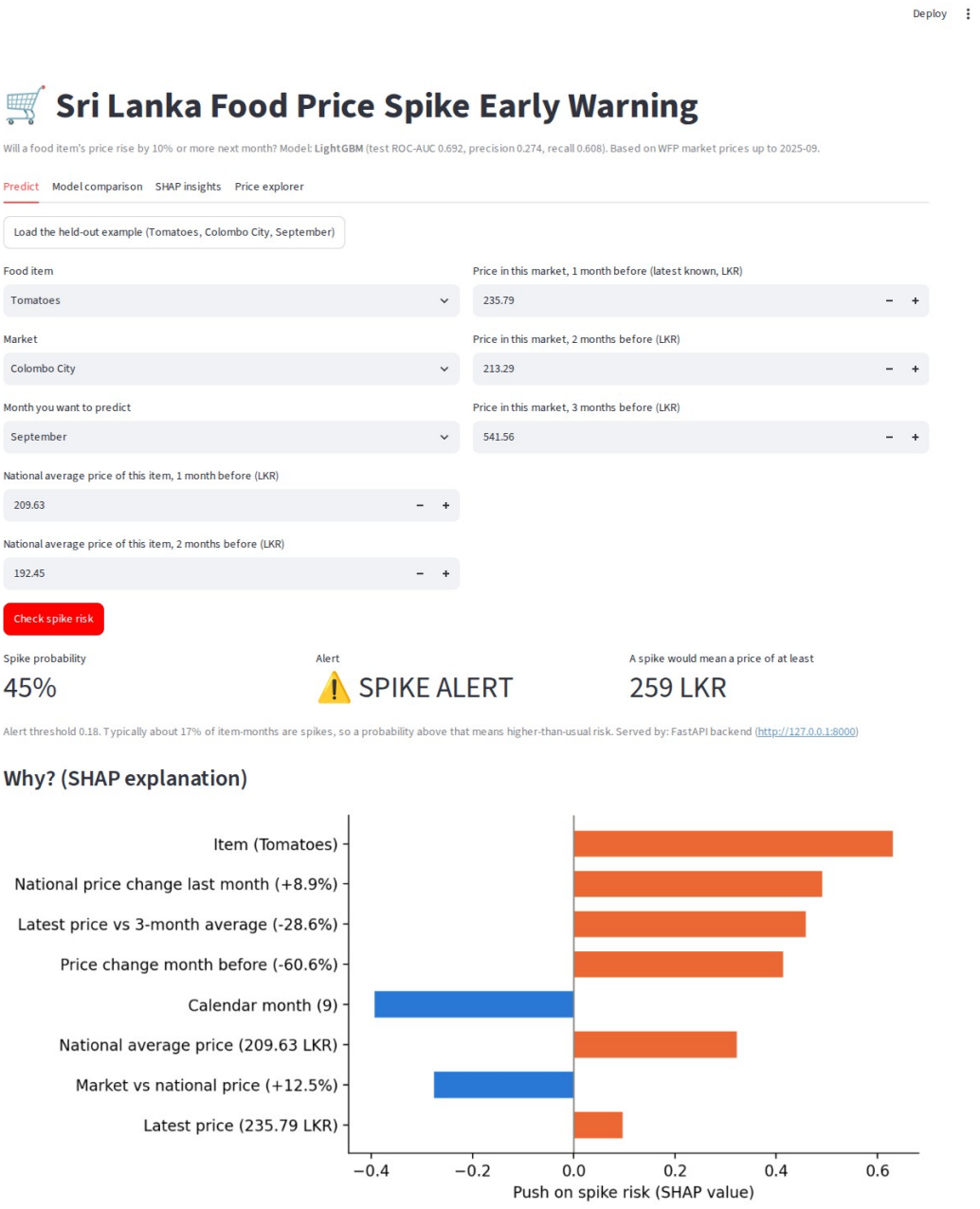


Figure 12: Dashboard result for the held-out example (served by the FastAPI backend)

7. Limitations and future work

- Only past prices are used. Weather, fuel and exchange-rate data, import data and festival calendars could improve the predictions.
- The gains over the item-rate baseline are modest and not statistically clear, and there is a single test period of nine months.
- Coverage before mid-2023 is thin, so most of the training signal comes from 2023 to 2025.
- A 10% threshold is a choice. A different threshold, or predicting the size of the price change (regression), may suit other uses.
- The file used is a copy of the WFP data with records up to September 2025. I did not independently verify it against the Humanitarian Data Exchange version, and I did not check how the WFP prices were collected.
- The alert threshold favours catching spikes (recall 0.61) over avoiding false alarms (precision 0.27). A different balance can be chosen depending on the cost of each kind of error.

8. Conclusion

Four tree-ensemble models were compared for early warning of food price spikes in Sri Lankan markets. LightGBM was selected on validation ROC-AUC (0.800) and reached ROC-AUC 0.692 on unseen 2025 data, only slightly above an item-based baseline and practically tied with XGBoost. SHAP showed that the model relies on sensible signals: the item, recent price falls (mean reversion), the national price trend and how the market compares with the national price. The model was deployed with a FastAPI backend and a Streamlit dashboard that explains every prediction.

9. Use of AI tools

AI tools (Claude) were used to help plan and write the code and the report draft. I reviewed and ran the code, and I understand the data preparation, the models, the evaluation, SHAP and the application. [Edit this paragraph so that it describes how you used the tools yourself.]

References

- World Food Programme. Sri Lanka - Food Prices. Humanitarian Data Exchange. https://data.humdata.org/dataset/wfp-food-prices-for-sri-lanka
- Lundberg, S. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. NeurIPS 2017.
- Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32.
- Chen, T. and Guestrin, C. (2016). XGBoost: A scalable tree boosting system. KDD 2016.
- Ke, G. et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. NeurIPS 2017.
- Prokhorenkova, L. et al. (2018). CatBoost: unbiased boosting with categorical features. NeurIPS 2018.